

省博物馆参观预约系统 · 数据库设计

省博物馆参观预约系统数据库设计

1. 文档说明

1.1 文档目的

本文档是 省博物馆参观预约系统 的正式数据库设计文档，用于指导后续 MySQL 建库建表、schema.sql 初始化脚本编写、Spring Boot 后端实体类设计、数据访问层设计、预约核心业务实现、并发测试和数据一致性验证。

1.2 设计依据

本文档根据以下资料整理：

- 项目要求分析.md
- 项目设计开发流程.md
- 需求拆分.md
- 系统设计文档初稿.md

1.3 设计定位

本数据库设计用于正式大作业系统，不复用之前环境验证 Demo 的简化表结构。

之前 Demo 只用于证明技术栈和本机环境可用，包括 Vue、Axios、Bootstrap、Spring Boot、MySQL、前后端联调和基础预约链路。正式系统需要根据完整需求重新设计数据库表、约束、索引和并发控制方案。

2. 数据库设计目标

数据库需要支撑游客端、管理员端和核心预约流程。

总体目标如下：

游客实名登录后，可以查看场馆信息、公告、预约日期和分时段，并提交预约；
管理员可以维护场馆信息、公告、预约活动、分时段名额，并查看预约记录和统计数据；
系统在多个游客同时预约同一时段时，不能出现超额预约、重复预约或名额数据不一致。

数据库重点解决的问题：

- 用户身份**：保存游客实名信息和管理员账号。
- 场馆展示**：保存场馆信息、开放时间、参观须知和公告。
- 预约配置**：保存可预约日期、预约开放时间、预约结束时间、单人预约上限和分时段名额。
- 预约记录**：保存预约编号、游客信息、参观日期、时段和二维码凭证内容。
- 并发安全**：通过数据库条件更新、唯一约束和事务控制防止超额和重复预约。
- 统计查询**：支持后台统计总名额、已预约人数、剩余名额和分时段预约人数。

3. 数据库基础约定

3.1 数据库类型

正式系统使用：

```
MySQL 8.0
```

推荐配置如下：

配置项	推荐值
存储引擎	InnoDB
字符集	utf8mb4
排序规则	utf8mb4_unicode_ci
主键类型	BIGINT UNSIGNED AUTO_INCREMENT
时间字段	DATETIME

选择 InnoDB 的原因：

- 支持事务
- 支持行级锁
- 支持外键约束
- 适合预约名额这种强一致性业务

3.2 命名规范

类型	规范	示例
数据库名	小写英文 + 下划线	museum_reservation
表名	小写英文 + 下划线	reservation_record
字段名	小写英文 + 下划线	booked_count
主键名	统一使用 id	id
创建时间	统一使用 created_at	created_at
更新时间	统一使用 updated_at	updated_at

3.3 时间字段约定

字段	说明
created_at	数据创建时间
updated_at	数据最后更新时间
booking_start	预约开放开始时间
booking_end	预约开放结束时间
visit_date	参观日期
start_time	入场时段开始时间
end_time	入场时段结束时间

时间规则：

- booking_end 必须晚于 booking_start
- end_time 必须晚于 start_time
- 预约开放判断必须以后端服务器时间为准
- 前端时间显示只能作为提示，不能作为最终校验依据

3.4 状态值约定

3.4.1 通用启用状态

值	含义
1	启用
0	禁用

3.4.2 公告类型

值	含义
NORMAL	普通通知
EXHIBITION	特殊展览
CLOSE	闭馆通知
RULE	预约规则

3.4.3 预约活动状态

值	含义
DRAFT	草稿，暂不开放
OPEN	启用，游客端可按时间规则预约
CLOSED	已关闭，不允许预约
DISABLED	禁用，不允许预约

说明：

活动是否未开始或已结束主要由当前时间与 booking_start、booking_end 对比得出；
status 主要表示管理员是否手动启用该预约活动。

3.4.4 预约记录状态

值	含义
SUCCESS	预约成功
CANCELLED	已取消，预留状态

课程大作业初期可以只实现 SUCCESS，不实现取消功能。

4. 数据表总览

正式系统建议设计 8 张表。

序号	表名	中文名称	是否核心表	主要作用
1	admin_user	管理员表	是	保存后台管理员账号
2	visitor	游客表	是	保存实名游客信息
3	museum_info	场馆信息表	是	保存博物馆开放信息和参观须知
4	notice	通知公告表	是	保存公告、特殊展览和闭馆通知
5	reservation_activity	预约活动表	是	保存预约日期、预约开放时间和每日规则
6	reservation_slot	预约时段表	是	保存分时时段、总名额和已预约人数
7	reservation_record	预约记录表	是	保存游客预约成功记录
8	request_log	请求日志表	可选	保存预约请求统计和系统状态数据

5. 表关系设计

5.1 关系总览

admin_user	独立表，用于后台登录
museum_info	独立表，用于游客端和后台展示场馆信息
notice	独立表，用于公告管理
visitor 1 —— N reservation_record	一个游客可以有多条预约记录
reservation_activity 1 —— N reservation_slot	一个预约活动可以包含多个分时时段
reservation_activity 1 —— N reservation_record	一个预约活动可以产生多条预约记录
reservation_slot 1 —— N reservation_record	一个预约时段可以产生多条预约记录
request_log	独立表，用于记录请求统计，可选

5.2 核心预约链路关系

核心预约链路依赖以下表：

visitor
reservation_activity

reservation_slot
reservation_record

提交预约时的数据流如下：

1. 根据身份证号找到或创建 visitor。
2. 根据 slot_id 找到 reservation_slot。
3. 根据 activity_id 找到 reservation_activity。
4. 校验预约活动状态、预约时间、时段状态和剩余名额。
5. 原子更新 reservation_slot.booked_count。
6. 插入 reservation_record。
7. 返回预约编号和二维码内容。

6. 详细表结构设计

6.1 管理员表 admin_user

6.1.1 表作用

admin_user 用于保存后台管理员账号信息，支撑管理员登录和后台权限判断。

6.1.2 字段设计

字段名	类型	是否为空	默认值	说明
id	BIGINT UNSIGNED	否	自增	主键
username	VARCHAR(50)	否	无	管理员用户名
password	VARCHAR(100)	否	无	管理员密码，后续可存储加密值
role	VARCHAR(30)	否	ADMIN	管理员角色
status	TINYINT	否	1	状态，1 启用，0 禁用
created_at	DATETIME	否	当前时间	创建时间
updated_at	DATETIME	否	当前时间	更新时间

6.1.3 约束与索引

类型	字段	说明
主键	id	唯一标识管理员
唯一索引	username	防止管理员用户名重复
普通索引	status	便于查询启用管理员

6.1.4 设计说明

- 系统初始化时预置一个管理员账号。
- 大作业阶段可使用简单登录逻辑，不强制引入 Spring Security。
- 如果时间允许，密码可改为加密存储。

6.2 游客表 visitor

6.2.1 表作用

visitor 用于保存游客实名信息。游客登录和预约记录都依赖该表。

6.2.2 字段设计

字段名	类型	是否为空	默认值	说明
id	BIGINT UNSIGNED	否	自增	主键
name	VARCHAR(50)	否	无	游客姓名
id_card	VARCHAR(18)	否	无	身份证号
phone	VARCHAR(20)	否	无	手机号
created_at	DATETIME	否	当前时间	创建时间
updated_at	DATETIME	否	当前时间	更新时间

6.2.3 约束与索引

类型	字段	说明
主键	id	唯一标识游客
唯一索引	id_card	一个身份证号对应一个游客
普通索引	phone	支持按手机号查询

6.2.4 设计说明

- 身份证号是游客实名身份的核心标识。
- 游客首次登录时，如果身份证号不存在，则创建游客记录。
- 游客再次登录时，如果手机号或姓名发生变化，可更新 name 和 phone。
- 身份证号和手机号格式主要由后端校验，数据库通过长度和非空约束辅助保证。

6.3 场馆信息表 museum_info

6.3.1 表作用

museum_info 用于保存博物馆基础信息，包括场馆名称、地址、开放时间、参观须知和预约规则。

6.3.2 字段设计

字段名	类型	是否为空	默认值	说明
id	BIGINT UNSIGNED	否	自增	主键
name	VARCHAR(100)	否	无	场馆名称
address	VARCHAR(255)	否	无	场馆地址
open_info	TEXT	否	无	开放时间说明
rules	TEXT	否	无	参观须知和预约规则

status	TINYINT	否	1	状态，1 正常，0 停用
created_at	DATETIME	否	当前时间	创建时间
updated_at	DATETIME	否	当前时间	更新时间

6.3.3 约束与索引

类型	字段	说明
主键	id	唯一标识场馆信息
普通索引	status	查询正常展示的场馆信息

6.3.4 设计说明

- 系统至少初始化一条场馆信息。
- 游客端首页读取状态为启用的场馆信息。
- 管理员后台可以修改该表内容。
- 如果系统只维护一个博物馆，可以固定使用 id = 1 的记录。

6.4 通知公告表 notice

6.4.1 表作用

notice 用于保存游客端展示的通知公告，例如预约规则、特殊展览、临时闭馆等。

6.4.2 字段设计

字段名	类型	是否为空	默认值	说明
id	BIGINT UNSIGNED	否	自增	主键
title	VARCHAR(100)	否	无	公告标题
content	TEXT	否	无	公告内容
type	VARCHAR(30)	否	NORMAL	公告类型
enabled	TINYINT	否	1	是否启用，1 启用，0 禁用
created_at	DATETIME	否	当前时间	创建时间
updated_at	DATETIME	否	当前时间	更新时间

6.4.3 约束与索引

类型	字段	说明
主键	id	唯一标识公告
普通索引	enabled	查询启用公告
普通索引	type	按公告类型筛选
普通索引	created_at	按发布时间排序

6.4.4 设计说明

- 游客端只展示 enabled = 1 的公告。
- 后台可以新增、编辑、启用、禁用公告。
- 公告列表默认按 created_at 倒序展示。

6.5 预约活动表 reservation_activity

6.5.1 表作用

reservation_activity 用于配置某一天的预约规则，包括参观日期、每日总名额、预约开放时间、预约结束时间、单人预约上限和活动状态。

6.5.2 字段设计

字段名	类型	是否为空	默认值	说明
id	BIGINT UNSIGNED	否	自增	主键
activity_name	VARCHAR(100)	否	无	预约活动名称
visit_date	DATE	否	无	参观日期
daily_capacity	INT UNSIGNED	否	无	当日预约总名额上限
person_limit	INT UNSIGNED	否	1	单人预约上限
booking_start	DATETIME	否	无	预约开放开始时间
booking_end	DATETIME	否	无	预约开放结束时间
status	VARCHAR(20)	否	DRAFT	活动状态
created_at	DATETIME	否	当前时间	创建时间
updated_at	DATETIME	否	当前时间	更新时间

6.5.3 约束与索引

类型	字段	说明
主键	id	唯一标识预约活动
唯一索引	visit_date	简化设计：同一参观日期只允许一个预约活动
普通索引	status	查询启用或关闭活动
普通索引	booking_start, booking_end	按预约时间范围查询

6.5.4 业务约束

必须满足：

```
booking_end > booking_start
daily_capacity > 0
person_limit > 0
```

时段容量约束：

同一预约活动下，所有启用时段的 total_capacity 总和不应超过 reservation_activity.daily_capacity。

6.5.5 设计说明

- 预约活动控制某一天是否开放预约。
- 游客是否能预约，需要同时满足活动状态、预约时间和时段状态。
- daily_capacity 是每日总上限，具体时段名额在 reservation_slot 中配置。
- 为降低大作业实现复杂度，正式系统初期采用“一天一个预约活动”的设计。

6.6 预约时段表 reservation_slot

6.6.1 表作用

reservation_slot 是数据库中最关键的表之一，用于保存某个预约活动下的分时时段、时段总名额、已预约人数和并发控制版本号。

该表直接影响：

- 剩余名额展示
- 是否允许预约
- 高并发下是否超额
- 预约统计是否准确

6.6.2 字段设计

字段名	类型	是否为空	默认值	说明
id	BIGINT UNSIGNED	否	自增	主键
activity_id	BIGINT UNSIGNED	否	无	所属预约活动 ID
visit_date	DATE	否	无	参观日期，冗余保存便于查询
slot_name	VARCHAR(50)	否	无	时段名称，例如 09:00-11:00
start_time	TIME	否	无	入场开始时间
end_time	TIME	否	无	入场结束时间
total_capacity	INT UNSIGNED	否	无	时段总名额
booked_count	INT UNSIGNED	否	0	已预约人数
enabled	TINYINT	否	1	是否启用，1 启用，0 禁用
version	INT UNSIGNED	否	0	版本号，用于并发控制
created_at	DATETIME	否	当前时间	创建时间
updated_at	DATETIME	否	当前时间	更新时间

6.6.3 约束与索引

类型	字段	说明
主键	id	唯一标识预约时段

外键	activity_id	关联 reservation_activity.id
唯一索引	activity_id, start_time, end_time	防止同一活动下重复配置相同时段
普通索引	activity_id	查询活动下所有时段
普通索引	visit_date, enabled	游客端按日期查询可预约时段

6.6.4 业务约束

必须满足：

```
end_time > start_time
total_capacity > 0
booked_count >= 0
booked_count <= total_capacity
```

后台调整名额时必须满足：

```
新的 total_capacity >= 当前 booked_count
```

剩余名额计算：

```
remaining = total_capacity - booked_count
```

6.6.5 设计说明

- booked_count 是预约成功人数的快速统计字段。
- 预约成功时，必须先通过数据库条件更新增加 booked_count，再插入预约记录。
- version 可用于乐观锁或辅助记录并发更新次数。
- 游客端显示剩余名额时，可以由 total_capacity - booked_count 得出。

6.7 预约记录表 reservation_record

6.7.1 表作用

reservation_record 用于保存游客预约成功后的记录，是游客查看“我的预约”、管理员查看名单、导出预约名单、生成二维码凭证和数据一致性核对的核心表。

6.7.2 字段设计

字段名	类型	是否为空	默认值	说明
id	BIGINT UNSIGNED	否	自增	主键
reservation_no	VARCHAR(50)	否	无	预约编号
visitor_id	BIGINT UNSIGNED	否	无	游客 ID
activity_id	BIGINT UNSIGNED	否	无	预约活动 ID
slot_id	BIGINT UNSIGNED	否	无	预约时段 ID

id_card	VARCHAR(18)	否	无	身份证号，冗余保存便于查询和导出
phone	VARCHAR(20)	否	无	手机号，冗余保存便于查询和导出
visit_date	DATE	否	无	参观日期
slot_name	VARCHAR(50)	否	无	预约时段名称
status	VARCHAR(20)	否	SUCCESS	预约状态
qr_content	VARCHAR(255)	否	无	二维码内容
created_at	DATETIME	否	当前时间	预约创建时间

6.7.3 约束与索引

类型	字段	说明
主键	id	唯一标识预约记录
唯一索引	reservation_no	保证预约编号唯一
唯一索引	id_card, visit_date	防止同一身份证同一天重复预约
外键	visitor_id	关联 visitor.id
外键	activity_id	关联 reservation_activity.id
外键	slot_id	关联 reservation_slot.id
普通索引	slot_id, status	统计某时段成功预约人数
普通索引	visitor_id	查询游客预约记录
普通索引	visit_date, slot_id	后台按日期和时段筛选
普通索引	created_at	按预约时间排序

6.7.4 业务约束

本系统初期采用以下规则：

同一身份证同一天只能预约一次。

对应唯一约束为：

id_card + visit_date

如果后续允许同一天预约多个不同时段，可以调整为：

id_card + slot_id

6.7.5 设计说明

- reservation_no 用于游客凭证和后台查询。
- id_card、phone、slot_name 冗余保存，是为了导出名单和历史记录展示更方便。
- 预约记录插入必须和时段名额扣减处于同一个事务中。
- 如果插入预约记录失败，前面增加的 booked_count 必须回滚。

6.8 请求日志表 request_log

6.8.1 表作用

request_log 用于记录预约接口或关键接口的请求情况，支撑后台系统状态查看、压测结果统计和演示说明。

该表为可选表，但建议保留，因为它有助于展示系统状态和性能测试结果。

6.8.2 字段设计

字段名	类型	是否为空	默认值	说明
id	BIGINT UNSIGNED	否	自增	主键
request_type	VARCHAR(50)	否	无	请求类型，例如 RESERVATION
success	TINYINT	否	无	是否成功，1 成功，0 失败
message	VARCHAR(255)	是	空	返回消息或失败原因
created_at	DATETIME	否	当前时间	请求时间

6.8.3 约束与索引

类型	字段	说明
主键	id	唯一标识请求日志
普通索引	request_type	按请求类型统计
普通索引	success	统计成功和失败请求
普通索引	created_at	按时间范围查询

6.8.4 设计说明

- 可用于后台系统状态页面展示预约接口累计请求数。
- 可用于压测后统计成功数、失败数和最近请求时间。
- 如果开发时间紧张，可以先不实现该表对应功能，但表设计保留。

7. 核心约束汇总

7.1 主键约束

所有业务表均使用 id 作为主键。

```
id BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY
```

优点：

- 便于后端统一处理实体 ID。
- 便于建立外键关联。
- 便于后续分页、查询和排序。

7.2 唯一约束

表名	唯一字段	作用
admin_user	username	防止管理员用户名重复
visitor	id_card	防止同一身份证创建多个游客
reservation_activity	visit_date	简化设计：同一天只配置一个预约活动
reservation_slot	activity_id, start_time, end_time	防止同一活动下重复配置相同时段
reservation_record	reservation_no	防止预约编号重复
reservation_record	id_card, visit_date	防止同一身份证同一天重复预约

最重要的唯一约束是：

```
reservation_record(id_card, visit_date)
```

该约束直接用于防止重复预约。

7.3 外键关系

从表	字段	主表	字段	说明
reservation_slot	activity_id	reservation_activity	id	一个活动包含多个时段
reservation_record	visitor_id	visitor	id	一条预约记录属于一个游客
reservation_record	activity_id	reservation_activity	id	一条预约记录属于一个活动
reservation_record	slot_id	reservation_slot	id	一条预约记录属于一个时段

7.4 检查约束

MySQL 8.0 支持 CHECK 约束，正式建表时可以加入以下约束。

表名	约束内容	说明
reservation_activity	daily_capacity > 0	每日总名额必须大于 0
reservation_activity	person_limit > 0	单人预约上限必须大于 0
reservation_activity	booking_end > booking_start	预约结束时间必须晚于开始时间
reservation_slot	total_capacity > 0	时段总名额必须大于 0
reservation_slot	booked_count <= total_capacity	已预约人数不能超过总名额
reservation_slot	end_time > start_time	入场结束时间必须晚于开始时间

说明：

```
数据库约束用于兜底，后端业务层仍然必须提前校验并返回友好错误信息。
```

8. 并发预约与数据一致性设计

8.1 核心风险

预约系统最大风险是多个游客同时抢同一个时段名额。

错误实现方式如下：

1. 先查询剩余名额。
2. 如果剩余名额大于 0，则继续预约。
3. 普通更新 booked_count。

该方式在高并发下可能导致多个请求同时看到同一个剩余名额，从而出现超额预约。

8.2 推荐实现方式

本系统推荐使用：

数据库条件更新 + 唯一约束 + 事务控制

核心条件更新语句：

```
UPDATE reservation_slot
SET booked_count = booked_count + 1,
    version = version + 1
WHERE id = ?
    AND enabled = 1
    AND booked_count < total_capacity;
```

判断方式：

影响行数 = 1：名额扣减成功，可以继续插入预约记录。
影响行数 = 0：名额已满或时段不可预约，返回预约失败。

8.3 事务边界

提交预约接口必须放入同一个数据库事务。

事务内步骤：

1. 校验身份证号和手机号格式。
2. 查询或创建 visitor。
3. 查询 reservation_slot 和 reservation_activity。
4. 校验活动状态、预约时间、时段状态。
5. 校验是否重复预约和是否超过单人上限。
6. 条件更新 reservation_slot.booked_count。
7. 插入 reservation_record。
8. 提交事务。

如果任一步失败，事务必须回滚。

必须避免以下错误状态：

- 名额增加了，但没有预约记录。
- 预约记录插入了，但名额没有增加。
- 并发请求导致成功记录数超过总名额。
- 同一身份证同一天出现多条成功预约记录。

8.4 重复预约控制

重复预约由两层保证：

第一层：后端业务查询，提前判断是否已有预约。
第二层：数据库唯一约束 reservation_record(id_card, visit_date) 兜底。

即使两个重复请求同时进入系统，数据库唯一约束也能保证最终只插入一条成功预约记录。

8.5 名额一致性核对

压测或功能测试完成后，需要核对：

reservation_slot.booked_count = reservation_record 中对应时段 SUCCESS 记录数
reservation_slot.booked_count <= reservation_slot.total_capacity

核对 SQL 思路：

```
SELECT
    s.id AS slot_id,
    s.slot_name,
    s.total_capacity,
    s.booked_count,
    COUNT(r.id) AS success_record_count
FROM reservation_slot s
LEFT JOIN reservation_record r
    ON s.id = r.slot_id
    AND r.status = 'SUCCESS'
GROUP BY s.id, s.slot_name, s.total_capacity, s.booked_count;
```

验收标准：

booked_count 必须等于 success_record_count。
booked_count 必须小于等于 total_capacity。

9. 常用统计查询设计

9.1 查询游客总数

```
SELECT COUNT(*) AS visitor_count
FROM visitor;
```

9.2 查询预约总数

```
SELECT COUNT(*) AS reservation_count
FROM reservation_record
WHERE status = 'SUCCESS';
```

9.3 查询各日期预约统计

```
SELECT
    visit_date,
    COUNT(*) AS reservation_count
FROM reservation_record
WHERE status = 'SUCCESS'
GROUP BY visit_date
ORDER BY visit_date;
```

9.4 查询各时段预约统计

```
SELECT
    s.visit_date,
    s.slot_name,
    s.total_capacity,
    s.booked_count,
    s.total_capacity - s.booked_count AS remaining_count
FROM reservation_slot s
ORDER BY s.visit_date, s.start_time;
```

9.5 查询后台预约名单

```
SELECT
    r.reservation_no,
    v.name,
    r.id_card,
    r.phone,
    r.visit_date,
    r.slot_name,
    r.created_at
FROM reservation_record r
JOIN visitor v ON r.visitor_id = v.id
WHERE r.status = 'SUCCESS'
ORDER BY r.created_at DESC;
```

9.6 查询我的预约记录

```
SELECT
    reservation_no,
    visit_date,
    slot_name,
    status,
    qr_content,
    created_at
FROM reservation_record
WHERE id_card = ?
    AND phone = ?
ORDER BY created_at DESC;
```

10. 初始化数据设计

10.1 管理员初始化

正式系统需要预置一个管理员账号。

示例：

```
username: admin
password: admin123
role: ADMIN
status: 1
```

说明：

如果后续实现密码加密，初始化密码需要存储加密后的值。

10.2 场馆信息初始化

需要初始化一条省博物馆基础信息。

建议内容包括：

- 场馆名称
- 场馆地址
- 开放时间
- 预约规则
- 参观须知

10.3 公告初始化

可以初始化 1 到 3 条公告，用于游客端展示和后台管理演示。

示例类型：

- 普通预约通知
- 特殊展览通知
- 临时闭馆或入馆提醒

10.4 预约活动初始化

建议初始化未来 2 到 3 天的预约活动。

每个活动包含：

- 参观日期
- 每日总名额
- 预约开始时间
- 预约结束时间
- 单人预约上限
- 活动状态

10.5 预约时段初始化

每个预约活动建议初始化 2 到 4 个时段。

示例：

--	--	--	--

时段名称	入场开始时间	入场结束时间	名额
上午场	09:00	11:00	50
中午场	11:00	13:00	50
下午场	13:00	15:00	50
傍晚场	15:00	17:00	50

说明：

同一活动下所有时段名额之和不应超过该活动 daily_capacity。

11. 与后端实现的对应关系

数据库表	后端模块	前端页面
admin_user	管理员登录模块	管理员登录页
visitor	游客登录模块	游客登录页
museum_info	场馆信息模块	游客首页、后台场馆信息页
notice	公告管理模块	游客公告区、后台公告管理页
reservation_activity	预约活动模块	后台活动配置页
reservation_slot	时段名额模块、预约核心模块	游客预约页、后台时段配置页
reservation_record	预约记录模块	我的预约页、后台预约名单页
request_log	系统状态模块	后台系统状态页

12. 数据库设计与评分点对应关系

评分点	数据库设计支撑
功能完整性	8 张表覆盖游客端和管理员端主要功能
高并发与稳定性	reservation_slot 条件更新控制并发名额
数据一致性	唯一约束、事务、booked_count 核对保证不超额不重复
虚拟机部署	使用 MySQL 8.0，适合部署到 Linux 虚拟机
文档规范	本文档可作为系统设计文档和数据库说明的补充材料

13. 后续开发建议

完成本文档后，下一步应生成：

schema.sql

schema.sql 应包含：

- 建库语句
- 8 张表的建表语句
- 主键、外键、唯一索引和普通索引
- 必要的 CHECK 约束
- 管理员、场馆、公告、预约活动和时段的初始化数据

推荐后续顺序：

1. 根据本文档生成 `schema.sql`。
2. 创建正式 Spring Boot 后端项目。
3. 配置 MySQL 数据源。
4. 编写实体类、DTO、Repository 或 Mapper。
5. 优先实现预约核心事务逻辑。
6. 编写并发测试脚本验证不超额和不重复。

当前数据库设计已经可以支撑正式项目开始后端开发。